SAGAPI v2.0 — Formulación MILP actualizada

Programación y reprogramación reactiva de un taller metalmecánico con averías, personal y horizonte rodante

Contenido

1. Alcance y convenciones
2. Conjuntos
3. Parámetros
4. Variables
5. Restricciones
6. Cota superior de tiempo y validez de los big-M
7. Personal (25 trabajadores, turno único de 8 h)
8. Objetivos y jerarquía
9. Horizonte rodante, averías y recuperación
10. Correspondencia formulación–código–prueba
11. Tamaño del modelo
12. Validación matemática
13. Limitaciones de la formulación

1. Alcance y convenciones

El modelo decide en cada **instante de decisión** t_0 (llegada de pedidos o avería) la unidad de recurso y el instante de inicio de las operaciones que aún no comenzaron. Lo ejecutado queda fijo (estrategia predictiva-reactiva, horizonte rodante por eventos).

- **Eje de tiempo.** Horas productivas acumuladas desde el 1-ene-2025. Solo cuentan los tramos de jornada de los días productivos de `09_CALENDARIO` (08:00–12:00 y 13:00–17:00, parámetro `jornada_productiva`). Un día productivo aporta $h_{dia} = 8$ h. Por construcción, ningún trabajador ni máquina opera fuera del turno único de 8 h.
- **Naturaleza de los datos.** Pedidos, averías y habilidades del personal son SIMULADOS; los tiempos de ruta son ESTIMADOS (no cronometrados); las penalidades y los costos son ESTIMADOS. La dotación de 25 trabajadores, el turno único y las 8 h por día son datos REALES declarados por la empresa.
- **Implementación.** `01_CODIGO_PYTHON/sagapi/modelo/milp.py` (modelo), `optimizacion.py` (procedimiento lexicográfico), `solver.py` (HiGHS vía `scipy.optimize.milp`, API pública, un hilo, semilla 0).

2. Conjuntos

Símbolo	Definición	Código
J	pedidos del subproblema (activos: liberados y no terminados)	`sub.jobs`
$O_j = (o_{j1}, \ldots, o_{jn_j})$	operaciones del pedido j en orden de ruta	`inst.job_ops[j]`
$O^R = O^P \cup O^I$	operaciones que se deciden: pendientes e interrumpidas	`sub.free`
O^F	operaciones fijas: completadas, en curso en una unidad sana o congeladas	`sub.fixed`
M	unidades de recurso; un recurso con cantidad $k > 1$ se expande en $m\#1, \ldots, m\#k$	`inst.machines`
$M_o \subseteq M$	unidades elegibles para o (principal y alternativas de `07_ELEGIBILIDAD`)	`sub.eligR[o]`
$O^N \subseteq O^R$	operaciones no interrumpibles (`interrumpible_en_pausas = 0`, p. ej. curado en horno)	`ops[o]["interrumpible"]`
W	ventanas futuras de indisponibilidad (m_w, a_w, b_w) con $a_w > t_0$	`sub.windows`
$\mathscr{P}$	pares (a, b) de operaciones de pedidos distintos con $M_a \cap M_b \neq \varnothing$	`mdl.y`

3. Parámetros

Símbolo	Definición	Origen
p_{om}	tiempo de proceso (h): $(\lceil q/L \rceil s + q\,u)/60 \cdot f_m$; en reprogramación, tiempo restante de la operación interrumpida	`06_RUTAS`, `07_ELEGIBILIDAD` (ESTIMADO)
r_j, d_j	liberación y vencimiento en horas productivas (fecha + `hora_liberacion` / `hora_vencimiento`)	`20_PEDIDOS_SIM`, `11_PARAMETROS`
w_j	peso: valor del pedido / valor medio	CALCULADO (precio medio por familia: ESTIMADO)
P_j	penalidad fija estimada: fracción de la familia × valor	`01_FAMILIAS` (ESTIMADO)
α_j, $\bar{g}_j$	penalidad diaria y tope (regla `PENALIDAD_DIARIA_CON_TOPE`)	CALCULADO
a_m	instante desde el que la unidad m está libre (fijas y averías en curso)	`sub.mavail`
lb_o	cota inferior de inicio: $\max(t_0, r_j, C_{\text{pred fija}})$	`sub.lbR`
S_o^0, m_o^0	inicio y unidad vigentes (solo al reprogramar)	`sub.S0`, `sub.m0`
L	longitud del tramo continuo: 4 h (`TRAMO`, no cruza refrigerio ni cierre) u 8 h (`JORNADA`)	`lote_no_cruza`
β	costo equivalente de una reasignación (h)	`beta_reasignacion` = 8
c_o	personas requeridas por la operación	`trabajadores_requeridos` (=1, ESTIMADO)
UB	cota superior de tiempo (sección 6)	`Subproblema.upper_bound_time`

4. Variables

Variable	Tipo	Significado
$S_o \in [lb_o, UB]$	continua	inicio de $o \in O^R$
$x_{om} \in \{0,1\}$	binaria	o se asigna a la unidad $m \in M_o$
$y_{ab} \in \{0,1\}$	binaria	a precede a b si comparten unidad ($(a,b) \in \mathcal{P}$)
$z_{ow} \in \{0,1\}$	binaria	o se procesa después de la ventana futura w
$k_o \in \mathbb{Z}_{\geq 0}$	entera	índice del tramo continuo en que se procesa $o \in O^N$
$T_j \geq 0$	continua	tardanza del pedido j (h)
$U_j \in \{0,1\}$	binaria	el pedido j termina después de d_j
$g_j \geq 0$, $v_j \in \{0,1\}$	mixtas	penalidad diaria con tope (solo regla diaria)

Variable	Tipo	Significado
$\delta_o^+, \delta_o^- \geq 0$	continuas	adelanto y atraso respecto de S_o^0 (solo al reprogramar)

Notación: $C_o = S_o + \sum_{m \in M_o} p_{om} x_{om}$ y $C_j = C_{o_{jn_j}}$.

5. Restricciones

R1 — asignación única. $\sum_{m \in M_o} x_{om} = 1 \quad \forall o \in O^R$.

R2 — disponibilidad de la unidad. $S_o \geq a_m \, x_{om} \quad \forall o, m \in M_o$: $a_m > lb_o$ (solo se escribe cuando no es redundante).

R3 — precedencia de ruta. $S_b \geq S_a + \sum_m p_{am} x_{am}$ para operaciones consecutivas $a \to b$ del mismo pedido, ambas en O^R. Si el predecesor está fijo, su fin entra en lb_b; la liberación r_j también entra en lb_o.

R4–R5 — capacidad de cada unidad (disyunción). Para $(a,b) \in \mathscr{P}$ y $m \in M_a \cap M_b$:

$$S_b \geq S_a + p_{am} - M_{ab}\,(3 - x_{am} - x_{bm} - y_{ab}), \qquad M_{ab} = UB + p_{am} - lb_b$$
$$S_a \geq S_b + p_{bm} - M_{ba}\,(2 - x_{am} - x_{bm} + y_{ab}), \qquad M_{ba} = UB + p_{bm} - lb_a$$

Con $x_{am} = x_{bm} = 1$, $y_{ab} = 1$ impone a antes de b y $y_{ab} = 0$ lo contrario. Con alguna $x = 0$ ambas filas quedan inactivas porque $S_b - S_a \geq lb_b - UB = p_{am} - M_{ab}$. Las operaciones del mismo pedido no generan pares: su cadena de precedencia ya las separa.

R6 — ventanas futuras de indisponibilidad. Para $w = (m_w, a_w, b_w) \in W$ y o con $m_w \in M_o$, $lb_o < b_w$:

$$S_o + p_{om_w} \leq a_w + M_w(1 - x_{om_w}) + M_w \, z_{ow}, \qquad S_o \geq lb_o + (b_w - lb_o)\, z_{ow}, \qquad M_w = UB + p_{om_w} - a_w$$

La operación termina antes de a_w o empieza después de b_w. **Corrección v2.0:** v1.4 usaba $M_w = UB$, que no relaja la fila cuando $a_w < p_{om}$ y S_o se acerca a UB (recortaba soluciones factibles). Las ventanas que ya empezaron en t_0 se recogen en a_m (R2). Los mantenimientos planificados se conocen desde el inicio y entran como ventanas futuras en todos los eventos.

R7–R8 — tardanza e indicador de atraso. $T_j \geq C_j - d_j$; $C_j - d_j \leq M_j U_j$ con $M_j = \max(1, UB + \max p - d_j)$.

R9 — penalidad diaria con tope (solo regla `PENALIDAD_DIARIA_CON_TOPE`). $g_j \geq \alpha_j T_j - M_j^g v_j$, $g_j \geq \bar{g}_j \, v_j$, $M_j^g = \alpha_j(UB + \max p)$. Como g_j se minimiza, $g_j = \min(\alpha_j T_j, \bar{g}_j)$.

R10 — desviación respecto del programa vigente. $S_o - \delta_o^+ + \delta_o^- = S_o^0$ para $o \in O^P$ con programa vigente.

R12 — operaciones no interrumpibles (nueva en v2.0). Para $o \in O^N$:

$$L\,k_o \leq S_o, \qquad S_o + \sum_m p_{om} x_{om} \leq L\,k_o + L, \qquad k_o \in \{\lfloor lb_o / L \rfloor, \ldots, \lceil UB/L \rceil + 1\}$$

La operación cabe completa en un tramo $[kL, (k+1)L)$: un ciclo de curado no se parte en el refrigerio ni en el cierre. Los límites de tramo son múltiplos de L porque cada día productivo aporta exactamente $h_{dia} = nL$ horas. Una operación con $p_{om} > L$ es un error de datos explícito.

R11 (v1.4) — eliminada. Reservaba personas por máquina sin variables de programación: no restringía el cronograma y daba una falsa sensación de cobertura laboral. El personal se trata en la sección 7.

6. Cota superior de tiempo y validez de los big-M

$$UB = \max(t_0,\ \max_m a_m,\ \max_o lb_o,\ \max_w b_w) + \sum_{o \in O^R} \max_{m \in M_o} p_{om} + \sum_{w \in W} (b_w - a_w) + L\,|O^N|$$

(y, al reprogramar, $UB \geq \max S^0 + \max p + 1$). Existe una solución con todos los inicios $\leq UB$: programar las operaciones una tras otra en cualquier orden compatible con las rutas a partir del máximo de las cotas, saltando las ventanas y desplazando a lo sumo L cada operación no interrumpible. Por tanto, imponer $S_o \leq UB$ no excluye el óptimo, y cada big-M de la tabla es el menor valor que deja la fila inactiva en todo el dominio $[lb, UB]$:

Fila	Big-M	Condición de inactividad
R4–R5	$UB + p_a - lb_b$	$S_b - S_a \geq lb_b - UB$
R6	$UB + p_{om} - a_w$	$S_o + p_{om} - a_w \leq UB + p_{om} - a_w$
R8	$\max(1, UB + \max p - d_j)$	$C_j - d_j \leq UB + \max p - d_j$
R9	$a_j(UB + \max p)$	$a_j T_j \leq a_j(UB + \max p)$

La prueba `test_R6_ventana_futura_y_bigM_valido` evalúa cada fila R6 en la esquina $S = UB$, $x = 0$, $z = 0$, y la enumeración exhaustiva (sección 9) detectaría cualquier big-M que recortara un programa factible.

7. Personal (25 trabajadores, turno único de 8 h)

Reglas laborales: **L1** cada operación (y cada tramo parcial ejecutado) tiene exactamente c_o trabajadores distintos; **L2** cada trabajador está habilitado para el recurso; **L3** ningún trabajador atiende dos intervalos solapados; **L4** carga diaria $\leq h_{dia}$; **L5** solo trabajadores existentes y no ausentes en el escenario.

Certificado de cobertura (condición suficiente). Sea $b_m = \max\{c_o : m \in M_o\}$. Se resuelve un flujo máximo fuente $\to$ unidad (capacidad b_m) $\to$ trabajador habilitado (1) $\to$ sumidero (1). Si el flujo vale $\sum_m b_m$, cada unidad recibe un equipo fijo de b_m personas distintas. *Proposición:* entonces todo programa que cumpla R1–R12 satisface L1–L5. *Demostración:* cada unidad procesa a lo sumo una operación a la vez (R4–R5) y su equipo es exclusivo, así que ningún trabajador está en dos intervalos a la vez (L3); el equipo está habilitado (L2) y tiene $b_m \geq c_o$ personas (L1); como el eje solo contiene horas de jornada y no hay solapes, la carga por día es $\leq h_{dia}$ (L4). ▫

Con la base actual el certificado se cumple (15 unidades, 25 trabajadores), también en los escenarios con 20 y 15 trabajadores. Las restricciones laborales son entonces redundantes y no se agregan al MILP, lo que reduce su tamaño sin perder factibilidad. **El resultado depende de la matriz de habilidades SIMULADA.**

Asignación de trabajadores (MILP posterior, equilibrio de carga). Sobre el programa ejecutado, con $q_{iw} \in \{0, 1\}$ para cada intervalo i y trabajador habilitado w:

$$\sum_{w \in W_i} q_{iw} = c_i; \qquad \sum_{i \in K} q_{iw} \leq 1 \ \ \forall w, \forall K \in \mathcal{K}_w; \qquad H_w = \sum_i p_i q_{iw},\ H_{\min} \leq H_w \leq H_{\max}$$

donde $\mathcal{K}_w$ son las cliques maximales del grafo de intervalos elegibles para w, obtenidas con un barrido; en un grafo de intervalos estas restricciones equivalen exactamente a «sin solape». Fases lexicográficas: $\min H_{\max}$, luego $\max H_{\min}$.

Sin certificado (p. ej. ausencias). Las reglas (FIFO, EDD, CONV) reservan trabajadores intervalo por intervalo (`EstadoPersonal.primer_inicio/reservar`), y el programa del MILP se repara: mismas unidades y secuencias, retrasando solo lo necesario. Si la reparación modifica algún objetivo, el

resultado se rotula `HEURISTICA_VERIFICADA` (nunca óptimo). Si una operación solo puede hacerse en recursos sin personal habilitado, el programa se detiene con un mensaje explícito.

8. Objetivos y jerarquía

Fase	Función	Unidad	Justificación
P1 (defecto)	$f_T = \sum_j w_j T_j$	h ponderadas	Medible directamente con fechas y horas; no depende de cláusulas contractuales inexistentes; prioriza los pedidos de mayor valor.
P2	$f_P = \sum_j P_j U_j$	S/ (ESTIMADO)	Desempata entre programas de igual tardanza ponderada a favor de menos penalidades estimadas.
P3 (solo al reprogramar)	$f_E = \sum_o (\delta_o^+ + \delta_o^-) + \beta \sum_o (1 - x_{o,m_o^0})$	h equivalentes	Estabilidad: entre programas igualmente buenos, el que menos cambia el plan comunicado a planta.

Las fases se resuelven en orden. En la fase k se agrega $f_i \le f_i^* (1+\tau) + \varepsilon$ para $i < k$, con $\tau = 10^{-6}$ y $\varepsilon = 10^{-4}$. **No se suman objetivos de unidades distintas.** Alternativas configurables en `28_CONFIG_MODELO` (`regla_objetivo`): `PENALIDAD_FIJA` (P1 = f_P, P2 = f_T; reproduce v1.4) y `PENALIDAD_DIARIA_CON_TOPE` (P1 = $\sum g_j$, P2 = f_T). Elegir f_T por defecto tiene un costo que se informa en los resultados: puede atrasar más pedidos que EDD aunque reduzca mucho la tardanza ponderada.

Estados de solución (nunca se confunden): `OPTIMO` (todas las fases con brecha $\le 10^{-5}$ y verificación algebraica), `FACTIBLE_LIMITE_TIEMPO`, `HEURISTICA_VERIFICADA` (solución de regla usada como respaldo, o reparación laboral), `SIN_SOLUCION_LIMITE_TIEMPO`, `INFACTIBLE`, `NO_ACOTADO`, `SOLUCION_INVALIDA`, `ERROR_EJECUCION`. Toda solución pasa además por el verificador independiente (`verificacion.py`), que no reutiliza el modelo.

9. Horizonte rodante, averías y recuperación

En cada evento t_0: O^C completadas ($C \le t_0$), excluidas; O^E en curso en unidad sana, fijas; O^I en curso en la unidad que falla, liberadas con tiempo restante; O^P pendientes, liberadas. Con horizonte congelado $H > 0$, una pendiente que empieza antes de $t_0 + H$ queda fija si su predecesor está fijo y ninguna ventana la cruza. **Política A** (conserva piezas): restante $= (q-k)\,uf +$ (lotes pendientes + preparación repetida)$\,sf +$ verificación. **Política B** (reinicia el lote): tiempo completo; las piezas hechas se reprocesan. El tramo ejecutado antes de la avería se registra como *parcial*: ocupa la unidad y su personal, pero no cuenta como producción. Así ninguna operación se produce dos veces. Controles en cada evento: ninguna operación iniciada (salvo la interrumpida) ni congelada vuelve a decidirse; ninguna completada cambia.

10. Correspondencia formulación–código–prueba

Elemento	Código	Prueba
R1–R2	`modelo/milp.py` bloques R1, R2	`test_R1_R2_asignacion_y_alternativa`
R3	bloque R3; `lbR`	`test_R3_precedencia_y_liberacion`
R4–R5, capacidad k	bloque R4-R5; `instancia.construir` (unidades)	`test_R4_R5_*`, enumeración con 2 unidades
R6	bloque R6	`test_R6_ventana_futura_y_bigM_valido`, enumeración
R7–R9	bloques R7-R8, R9	`test_R7_R8_*`, `test_R9_*`
R10, P3	bloque R10, `f_estabilidad`	`test_R10_*`, enumeración P3 por PL
R12	bloque R12; `reglas._ajustar_tramo`; `verificacion._cruza_tramo`	`test_R12_*`, enumeración
L1–L5	`personal.py`, `verificacion.verificar_personal`	`tests/test_personal.py`
Horizonte rodante	`estado.particionar`, `simulacion.simular`	`tests/test_rodante.py`
Lexicográfico y estados	`optimizacion.py`, `solver.py`	`tests/test_solver_estados.py`

11. Tamaño del modelo

$|x| = \sum_o |M_o|$; $|y| = |\mathcal{P}|$; $|z| \le |W| \cdot |O^R|$; $|k| = |O^N \cap O^R|$; filas R4–R5 $= 2\sum_{(a,b)} |M_a \cap M_b|$. El tamaño crece cuadráticamente con las operaciones libres que comparten unidades. Por eso no se resuelve un MILP anual monolítico (~54 000 binarias en v1.4): se resuelve un subproblema por evento. Los tamaños y tiempos medidos están en `04_RESULTADOS/Resultados_MILP/E5_rendimiento.csv` y en los registros de eventos.

12. Validación matemática

`tests/test_enumeracion.py` compara el MILP con la **enumeración exhaustiva** de todas las asignaciones y secuencias por unidad en 66 microinstancias aleatorias (70 generadas; 4 omitidas por no tener operaciones libres o por superar 7 operaciones): 2 jerarquías, recurso de 2 unidades, operaciones R12, averías actuales y simultáneas, ventanas futuras, políticas A y B y programación inicial. P1 y P2 se comparan con programas semiactivos (dominantes para objetivos regulares). P3 se compara mediante un PL de tiempos por asignación, secuencia y patrón de atraso, en los casos sin ventanas futuras ni R12. Todas las comparaciones coinciden (ver `06_AUDITORIA_Y_VALIDACION/Reporte_Pruebas`).

13. Limitaciones de la formulación

- Precedencia lineal sin lotes de transferencia: una operación empieza cuando termina todo el lote anterior (requiere datos de flujo real).
- Tiempos deterministas y estimados; duración de la avería conocida al ocurrir; ejecución perfecta entre eventos.
- El certificado laboral es suficiente, no necesario. Sin certificado, el tratamiento del personal es heurístico.
- Las penalidades son estimaciones; f_P no es un costo contractual verificado.

- Un programa factible para el modelo no está validado en planta.